

Software Evolution & Re- Engineering CS4118

Software Maintenance and Evolution

Lect 3

Contents

- Evolution vs Maintenance
- Software Evolution
- Software Maintenance
- Maintenance of COTS-Based Systems
- SOFTWARE EVOLUTION MODELS AND PROCESSES
- Software Reuse

Evolution vs Maintenance

- Another flaw in the human character is that everybody wants to build and nobody wants to do maintenance. --- Kurt Vonnegut, Jr.
- The concept of *software maintenance* means preventing software from failing to deliver the intended functionalities by means of bug fixing.
- The concept of *software evolution* means a continual change from a lesser, simpler, or worse state to a higher or better state.
- All support activities carried out *after* delivery of software are put under the category of *maintenance*.
- All activities carried out to effect changes in requirements are put under the category of *evolution*.
- Over time what evolves is not the software but our knowledge about a particular type of software

Software Maintenance

- Maintenance of software systems primarily means fixing bugs but preserving their functionalities
- Maintenance does not involve making major changes to the architecture of the system.
- Maintenance means keeping an installed system running with no change to its design
- Examples of Software Maintenance are:
 - fixing bugs; and/or improving the design of the system so that future changes to the system are less

Software Evolution

- Lehman and his collaborators formulated some observations and they introduced them as *laws of evolution*.
- Those laws are the results of studies of the evolution of large-scale proprietary or **closed source software (CSS)** systems.
- ***Continuing change***. Unless a system is continually modified to satisfy emerging needs of users, the system becomes increasingly less useful.
- ***Increasing complexity***. Unless additional work is done to explicitly reduce the complexity of a system, the system will become increasingly more complex due to maintenance-related changes.

Software Evolution

- ***Self-regulation.*** The evolution process is self-regulating in the sense that the measures of products and processes, that are produced during the evolution, follow close to *normal* distributions.
- ***Conservation of organizational stability.*** The average effective global activity rate on an evolving system is almost constant throughout the lifetime of the system. In other words, the average amount of additional effort needed to produce a new release is almost the same.

Software Evolution

- ***Conservation of familiarity***. As a system evolves all kinds of personnel, namely , developers and users must also evolves.
- ***Continuing growth***. As time passes, the functional content of a system is continually increased to satisfy user needs.
- ***Declining quality***. Unless the design of a system is diligently fine-tuned and adapted to new operational environments, the system's qualities will be perceived as declining over the lifetime of the system.
- ***Feedback system***. The system's evolution process involves multi-loop, multiagent, multi-level feedback among different kinds of activities.

Case Study

- Software Evolution of **Linux OS** using **Lehman's 7 Laws**
- **Homework**

Types of Software Maintenance Activities

- There are defects in delivered software applications, because defect removal and quality control processes are not perfect.
- ***Activities to make corrections.*** If there are discrepancies between the expected behavior of a system and the actual behavior, then some activities are performed to eliminate or reduce the discrepancies.
- ***Activities to make enhancements.*** A number of activities are performed to implement a change to the system, thereby changing the behavior or implementation of the system.

Types of Software Maintenance Activities(2)

- three subcategories of Enhancement activities are:
 - – enhancements that modify existing requirements;
 - – enhancements that create new requirements;
 - – enhancements that modify the implementation without changing the requirements.

Maintenance of COTS-Based Systems

- Many present-day software systems are built from components previously developed for other systems or to be reused in many systems.
- In this approach, new components are developed by combining...
 - Commercial off-the-shelf (COTS) components
 - custom-built (in-house) components
 - Open source software components.
- The components are obtained from a variety of sources and maintained by different vendors, possibly from different countries

SOFTWARE EVOLUTION MODELS AND PROCESSES

- The traditional view of the software life cycle has done a disservice to maintenance by depicting it solely as a single step at the end of the cycle.
- software maintenance should have its own software maintenance life cycle (SMLC) model.
- Three common features of the SMLC models found in the literature are:
 - understanding the code;
 - modifying the code; and
 - revalidating the code.

SOFTWARE EVOLUTION MODELS AND PROCESSES; Iterative Model

- *Iterative model*. Complete set of requirements for a system cannot be completely understood, or the developers do not know how to build the full system.
- systems are constructed in builds, each of which is a refinement of requirements of the previous build.
- A build is refined by considering feedback from users
- maintenance and evolution activities do not exist as distinct phases.
- They are closely intertwined.

SOFTWARE EVOLUTION MODELS AND PROCESSES; Change mini-cycle model

- *Change mini-cycle model*: five major phases
- change request
- Analyze and plan change
- implement change
- verify and validate
- documentation change

SOFTWARE EVOLUTION MODELS AND PROCESSES;

staged model of maintenance and evolution

- *staged model of maintenance and evolution*: primary objective is to improve the understanding of how long-lived software evolves.
- Four Stages:
- *Initial development*: detailed knowledge about the system is fresh
- *Evolution*: Significant changes involve higher cost and higher risk.
- *Servicing*: detailed knowledge is decreased, main focus on fixing bugs, adding small changes.
- *Phaseout*: minimal servicing of a system is not an option.
 - organization decides to replace the system.
 - too expensive to maintain the system.
 - newer solution available.

Software Reuse

- Software reuse means using existing software knowledge or artifacts during the development of a new system.
- Reuse is not constrained to source code fragments.
- *Data reuse*: involves a standardization of data formats.
- *Architectural reuse*: consists of standardizing a set of design and programming conventions dealing with the logical organization of software.
- *Design reuse*: deals with the reuse of abstract designs that do not include implementation details.
- *Program reuse*: deals with reusing running code

Reference Material

- Read Ch 1 of Software Evolution and Maintenance book